İSTİNYE ÜNİVERSİTESİ İSTANBUL

# Capstone Project Final Report

Submitted to
Faculty of Engineering and Natural Sciences
Department of Software Engineering

In partial fulfillment of the requirements for the degree
**BACHELOR of SCIENCE in SOFTWARE ENGINEERING**

## *"Ivy: A Style-Aware Multilingual Translator for Language Learning"*

**Student: Zaid Husam Eddin**
**ID Number: 220911692**
**Supervisor: Asst. Prof. Ragıp Mustafa Başbuğ**

Istanbul
May 2026

İSÜ İSTİNYE ÜNİVERSİTESİ İSTANBUL

# STUDENT DECLARATION

By submitting this report, as partial fulfillment of the requirements of the Capstone Project, the student promises on penalty of failure of the course that:

- I have given credit to and declared (by citation) any work that is not my own, including parts of the report that are copied or paraphrased from external sources.
- I have not received unpermitted aid for the project design, implementation, report, or presentation.
- I have not falsely assigned credit for work to another student and have not taken credit for work done by another student.

# Abstract

Foreign-language learners and travellers often need translations that go beyond a single, neutral rendering of the source sentence. The way a request is phrased in Turkish to a stranger differs sharply from how it would be said to a close friend, and the same is true across most living languages. Standard machine translation systems, despite producing fluent output, treat this dimension as noise and collapse every input to a generic register. This project presents Ivy, a style-aware multilingual translation system designed primarily for language learners. The system supports English, Turkish, Russian, and German across three style registers — formal, neutral, and casual — and combines a strong neutral translation baseline (Helsinki-NLP opus-mt-tc-big) with a large language model (Gemini 2.5 Flash) that performs targeted register restyling. A multilingual BERT-based style classifier provides a feedback signal indicating which register the output occupies. Beyond translation, Ivy integrates a learning loop: vocabulary saving with grammatical metadata, spaced-repetition flashcards using the SM-2 algorithm, AI-generated level-graded short stories with synchronized text-to-speech, and a community feature for sharing flashcard sets. On a held-out test set built from Tatoeba parallel data, the translation pipeline achieves a corpus BLEU of 44.85–62.78 across six language directions. The style classifier reaches 76.16% accuracy and a macro-F1 of 0.77 across 432 evaluation sentences in four languages. A close analysis of the confusion matrix reveals that the classifier rarely confuses formal with casual (zero errors out of 144 in this direction), suggesting that it has learned valid register dimensions but is conservatively calibrated toward the neutral class. Targeted register failures in Turkish — particularly the persistence of the formal copular suffix -dır in casual outputs — are addressed through a two-layer correction mechanism combining a deterministic regex detector and a focused LLM revision pass. The full system is deployed as a Flask-based single-page web application with user authentication, persistent vocabulary, and social flashcard sharing.

***Keywords:*** machine translation; style transfer; computer-assisted language learning; multilingual NLP; Turkish

# Table of Contents


Section 1: Introduction ..... 9
1.1. Background and Motivation ..... 9
1.2. Problem Statement ..... 10
1.3. Objectives ..... 10
1.4. Contributions ..... 11
1.5. Structure of the Report ..... 12
Section 2: Literature Review ..... 13
2.1. Neural Machine Translation: Foundations ..... 13
2.2. Multilingual Pre-trained Models ..... 13
2.3. The OPUS-MT Project and Open Translation Infrastructure ..... 14
2.4. Style, Register and Formality in Machine Translation ..... 15
2.5. Evaluation of Machine Translation Quality ..... 16
2.6. Computer-Assisted Language Learning and Spaced Repetition ..... 16
2.7. Synthesis and Identified Gaps ..... 17
Section 3: Methodology ..... 19
3.1. System Architecture ..... 19
3.2. The Translation Pipeline ..... 20
3.2.1. Base translation with opus-mt ..... 20
3.2.2. English chaining for non-English pairs ..... 21
3.2.3. Style restyling via Gemini 2.5 Flash ..... 21
3.2.4. The two-layer Turkish casual-copula correction ..... 21
3.3. Style Classifier ..... 23
3.3.1. Architecture and base model ..... 24
3.3.2. Training data construction ..... 24
3.3.3. Training procedure ..... 24
3.4. Word-Level Analysis ..... 25
3.5. Language-Learning Components ..... 25
3.5.1. Vocabulary persistence ..... 25
3.5.2. Spaced-repetition flashcards (SM-2) ..... 25
3.5.3. Short stories with synchronised audio (karaoke) ..... 26
3.5.4. Community: friends and snapshot-fork for public sets ..... 26
3.6. Implementation Details ..... 27
3.7. Evaluation Methodology ..... 27
3.7.1. Translation evaluation ..... 27
3.7.2. Style classifier evaluation ..... 28

3.7.3. Word-analysis evaluation....28
3.7.4. Summary of dataset construction....28
Section 4: Results....30
4.1. Translation Quality....30
4.2. Style Classification....31
4.3. Word-Level Analysis....33
4.4. Qualitative Findings from Cold-User Testing....34
Section 5: Discussion and Conclusion....36
5.1. Discussion of Results....36
5.2. Limitations....37
5.2.1. Evaluation methodology....37
5.2.2. Architectural limitations....38
5.2.3. Training data limitations....38
5.2.4. Scope limitations....38
5.3. Future Work....39
5.4. Lessons Learned....40
5.5. Conclusion....41
A.1. Style-restyling prompt (Gemini, cross-lingual)....44
A.2. Casual Turkish revise prompt (Gemini, single-language)....44
A.3. Style-evaluation data generation prompt....44
A.4. Key hyperparameters....45

# List of Tables

Table 1. Comparison of representative works against the present project (Ivy).

Table 2. Composition of evaluation datasets and their sources.

Table 3. Translation evaluation: BLEU and chrF++ on Tatoeba test pairs.

Table 4. Style classifier performance, per language.

Table 5. Style classifier overall confusion matrix (n = 432).

Table 6. Word-analysis evaluation summary (n = 100; Turkish).

# List of Figures

Fig. 1. Ivy system architecture (hybrid opus-mt + Gemini pipeline).

Fig. 2. Translation pipeline data flow with the two-layer Turkish casual-copula correction.

Fig. 3. Style classifier macro-F1 by language.

Fig. 4. Ivy main translation interface, showing the three-style comparison panel.

Fig. 5. Short-story generation and synchronized karaoke audio playback.

# List of Abbreviations

| | |
|---|---|
| **API** | Application Programming Interface |
| **BLEU** | Bilingual Evaluation Understudy (translation metric) |
| **BERT** | Bidirectional Encoder Representations from Transformers |
| **CALL** | Computer-Assisted Language Learning |
| **CEFR** | Common European Framework of Reference for Languages |
| **chrF** | Character n-gram F-score (translation metric) |
| **LLM** | Large Language Model |
| **MT** | Machine Translation |
| **NMT** | Neural Machine Translation |
| **NLP** | Natural Language Processing |
| **POS** | Part of Speech |
| **SM-2** | SuperMemo-2 spaced-repetition algorithm |
| **SPA** | Single-Page Application |
| **SRS** | Spaced Repetition System |
| **TTS** | Text-to-Speech |

# Section 1:
# Introduction

## 1.1. Background and Motivation

Five years ago I moved to Turkey, and learning Turkish stopped being a hobby and started being part of daily life. Going through that — making the early mistakes in public, sounding stiff to people who were only trying to chat, slowly figuring out which forms belonged where — shaped how I now think about translation. I sit at roughly B1 in Turkish today, B1 in Russian, and still firmly at the beginner end of German. Across all three, one thing has stayed constant: translation, as the average person encounters it on a phone or a web browser, is a very flat object. Type a sentence in, get a sentence out. The output is usually correct. It is also almost always wrong in the way that matters most for a learner: it sounds like nobody in particular speaking to nobody in particular.

This flatness is most obvious in languages whose grammar encodes social distance directly. In Turkish, the choice between *siz* and *sen* is not optional; it tells the listener whether you respect them. In Russian, *Вы* and *ты* do the same job, with similar social weight. In German, *Sie* and *du* are equally non-negotiable. A learner who pastes “Could you tell me what time it is?” into a translator will get a single answer back — and that answer will commit them, silently, to a particular register that may or may not be appropriate for the situation they are walking into. Get it wrong and you sound either arrogant or absurdly stiff. Mainstream translation tools do not warn you about this. They treat the choice as already made.

The motivation for this project grew directly from that gap. If I, as a language learner, repeatedly need a translation that is not only correct but situated — appropriate to whether I am writing to a professor, a stranger on the street, or a close friend — then surely a tool can be built that takes the situation as an input rather than guessing. That tool should also help me actually learn the language, not just hand me the answer and walk away. A translation I do not understand the structure of is a translation I will forget by tomorrow.

Ivy is the system this project builds in response. It is a style-aware multilingual translator for four languages — English, Turkish, Russian, and German — across three register levels (formal, neutral, casual), integrated with a small but functional set of language-learning tools: vocabulary saving with grammatical metadata, spaced-repetition flashcards, AI-generated short stories at chosen CEFR levels, and a community feature for sharing learning material. The name “Ivy” was chosen for the way the plant grows across whatever surface it

meets, adapting to any wall, fence, or window frame without losing its identity; a translator that crosses language barriers while adapting tone seemed to share the same character.

## 1.2. Problem Statement

Mainstream neural machine translation (NMT) systems are trained on parallel corpora that are heavily biased toward a single, register-neutral style — typically newswire and Europarl-style legislative text [1, 2]. While this produces fluent and accurate output for everyday content, it has two specific consequences that affect the language-learning use case directly. First, the output is register-collapsed: a user asking for a Turkish translation of an informal English request will receive a translation that is grammatically correct but socially inappropriate. Second, NMT output is opaque to the learner — the translated string carries no information about its constituent words, their grammatical role, or their relationship to the source. A learner sees the answer but not the structure that produced it.

Concretely, this project addresses three interlocking problems:

- **Register-controlled translation.** How can a translation system be designed such that the user specifies the desired register (formal / neutral / casual) and the output reliably reflects that register across English, Turkish, Russian, and German?
- **Register detection.** Given an arbitrary input sentence, can the register be classified automatically with sufficient reliability for both end-user feedback and downstream filtering?
- **Integration with learning.** Can register-aware translation be embedded in a self-contained learning environment — vocabulary, spaced repetition, level-graded reading, audio — so that the system contributes to acquisition rather than substituting for it?

## 1.3. Objectives

The objectives of this capstone project are:

1. Design and implement an end-to-end style-aware translation pipeline covering six language directions (en↔tr, en↔de, en↔ru), where the user explicitly selects the target register.
2. Train a multilingual style classifier (formal / neutral / casual) for the same four languages and evaluate its calibration on held-out data.

3. Build a language-learning environment around the translator: word-level analysis with grammatical metadata, persistent vocabulary, an SM-2 spaced-repetition scheduler, level-graded short stories with synchronized audio, and a basic community/sharing layer.
4. Quantitatively evaluate the system on translation quality (BLEU, chrF++), style classification (accuracy, macro-F1, per-class precision and recall), and word-level POS / lemma agreement.
5. Honestly document the limitations of each component, including where the evaluation methodology itself is a known weak point, and identify concrete directions for future work.

## 1.4. Contributions

The main contributions of this work are the following:

- **A hybrid translation architecture** combining Helsinki-NLP opus-mt-tc-big models for the underlying translation step with Google's Gemini 2.5 Flash as a register-restyling layer. Non-English pairs are handled by English-chaining when no direct opus-mt model is available. The architecture is presented and justified against the alternatives explored during development (a fine-tuned mT5 baseline and a Gemini-only direct pipeline).
- **A two-layer linguistic correction pattern** for handling targeted register failures in LLM output. The instance presented here addresses the persistence of the formal copular suffix *-dır* (and its allomorphs) in casual Turkish output. A regex-based detector triggers a focused Gemini revise pass when failure is detected, with a deterministic strip operation as a final fallback when the API is unavailable. The same defensive pattern generalises to other register markers and other languages.
- **An honest evaluation of the style classifier** that goes beyond a single accuracy number. The confusion-matrix analysis shows that the classifier has learned valid register dimensions (formal–casual confusion is essentially zero) but is conservatively biased toward the neutral class. This pattern is more diagnostic than the headline accuracy and is presented as a primary result rather than an embarrassing footnote.
- **A complete language-learning surface** around the translator, including word-level analysis, vocabulary persistence, spaced-repetition flashcards (SM-2), AI-generated

CEFR-graded short stories with synchronized text-to-speech, and a friend / public-set community layer. The system is deployed as a Flask single-page application with bcrypt authentication.

## 1.5. Structure of the Report

The remainder of this report is organised as follows. Section 2 reviews the relevant literature on neural machine translation, style transfer in text, multilingual language modelling, register and politeness in sociolinguistics, evaluation of machine translation output, and computer-assisted language learning. A comparative summary of the reviewed work and the gaps it leaves are presented at the end of that section. Section 3 describes the system methodology: the high-level architecture, the choice of components, the dataset construction process for both training and evaluation, the design of the style classifier, the design of the two-layer register-correction mechanism, and the design of the language-learning components. Section 4 presents the implementation and the quantitative evaluation results across all three measurement tasks (translation, style, word analysis), together with qualitative observations from a small cold-user trial. Section 5 discusses what these results mean, identifies limitations both methodological and architectural, and outlines concrete future work. References, an acknowledgement, and an appendix containing additional implementation details follow.

# Section 2:
# Literature Review

This section reviews the prior work most directly relevant to a style-aware multilingual translator integrated with a learning environment. The literature is organised into six clusters: the foundations of neural machine translation; multilingual pre-trained models; open translation infrastructure (the OPUS-MT project); style and register in machine translation; the evaluation of translation output; and computer-assisted language learning, including spaced repetition. The section closes with a comparison of representative work in tabular form and a discussion of the gaps these works leave open — gaps which motivate the design decisions presented in the methodology section that follows.

## 2.1. Neural Machine Translation: Foundations

Modern machine translation rests on the sequence-to-sequence framework introduced by Sutskever, Vinyals and Le [3] and the attention mechanism formalised by Bahdanau, Cho and Bengio [2]. The Transformer architecture proposed by Vaswani et al. [1] removed the recurrent backbone entirely and replaced it with stacked self-attention layers; this is now the de facto basis of every system referenced in this report, including the Helsinki-NLP opus-mt models used by Ivy. A persistent challenge for these models is open-vocabulary translation, since rare words and morphologically rich forms blow up the softmax. Sennrich, Haddow and Birch [4] solved this elegantly with byte-pair encoding (BPE), enabling translation systems to operate over a fixed vocabulary of subword units that compose into novel words at inference time. The Marian toolkit by Junczys-Dowmunt and colleagues [5] provides the fast C++ runtime in which most opus-mt models are trained.

These four works together constitute the technical baseline of neural machine translation as it stands today. They are not in themselves directly concerned with style, register, or pedagogy; rather they describe the substrate on which any modern translator must be built. Ivy uses opus-mt-tc-big variants for the base translation step in every supported language pair, inheriting all of these innovations transitively.

## 2.2. Multilingual Pre-trained Models

A second strand of work concerns the use of large pre-trained models to support translation and translation-adjacent tasks across many languages. BERT, introduced by Devlin et al. [6], demonstrated that bidirectional masked-language-model pre-training produces

representations that transfer well to downstream tasks. Its multilingual variant (mBERT) provides cross-lingual representations from a single shared model, and is the encoder used by Ivy's style classifier — fine-tuned on a small dataset of synthetic style-paired sentences in English, Turkish, German and Russian. Conneau et al. [7] later improved on mBERT with XLM-R, training on a much larger and cleaner multilingual corpus.

In parallel, the text-to-text framing of Raffel et al. [8] — where every task is recast as a string-to-string transformation — became influential. Its multilingual variant, mT5 [9], pre-trains on Common Crawl across 101 languages and is the model originally used as a baseline in the early stages of this project. Although mT5-base produced fluent style restyling on the training distribution, it required more careful fine-tuning than a one-semester project could afford, and the production-quality translations are now handled by opus-mt while mT5 is retained as a research baseline. The trajectory from mBERT through XLM-R to mT5 illustrates an important shift: multilingual pre-training has moved from "classification-style" encoders to genuinely generative models, which directly supports the kind of register-restyling work the rest of this report addresses.

More recently, large language models such as GPT-3 [27] have shown that scaling alone produces models capable of zero- and few-shot instruction following. Chain-of-thought prompting [28] further showed that careful framing of the prompt can elicit improved performance on reasoning tasks. The use of Google's Gemini 2.5 Flash for register restyling in Ivy fits squarely within this paradigm: a single LLM is given an explicit instruction ("rewrite this Turkish sentence in casual register") and produces a controlled output without per-language fine-tuning. The Gemini API's combination of low cost, fast latency and competent multilingual handling is what makes the hybrid pipeline economically and practically feasible at a student-project scale.

## 2.3. The OPUS-MT Project and Open Translation Infrastructure

The OPUS project, started by Tiedemann [10], collects parallel corpora across hundreds of languages and is the source of most of the training data used by the opus-mt translation models. The OPUS-MT system itself, introduced by Tiedemann and Thottingal [11], packages over a thousand pre-trained translation models into a uniform interface and provides Hugging Face checkpoints that can be loaded with a single line of Python. A more recent journal-length treatment in *Language Resources and Evaluation* [12] discusses the broader programme of democratising machine translation, particularly for low- and mid-resource languages. The

opus-mt-tc-big variants used by Ivy are trained on the Tatoeba Translation Challenge data, which makes them particularly well-suited to the kind of short, conversational sentences that learners typically translate.

These works are foundational to Ivy in a very direct sense: without OPUS-MT, a student-scale project could not credibly offer translation across six directions in four languages. The literature here is best understood as infrastructure rather than as a prior method to be improved upon.

## 2.4. Style, Register and Formality in Machine Translation

The most direct prior work to the central problem of this project — register-aware machine translation — comes from a small but focused literature. Niu, Martindale and Carpuat [17] introduced the task of Formality-Sensitive Machine Translation (FSMT), in which the system takes a desired formality level as an additional input alongside the source sentence. They proposed a method based on lexical formality scoring to control the formality level of output. Niu, Rao and Carpuat [18] extended this with multi-task neural models that learn to translate between styles within a single language while simultaneously performing cross-lingual translation. Pavlick and Tetreault [19] provided the underlying empirical groundwork by analysing formality in online communication and showing that it can be reliably scored at the sentence level.

The wider literature on style transfer in monolingual text is also relevant. Shen et al. [13] proposed style transfer over non-parallel data by aligning cross-style latent representations. Hu et al. [14] used a generative model with disentangled latent codes for controlled text generation, including for attributes like sentiment. Li et al. [15] proposed a simpler delete-retrieve-generate pipeline that performs surprisingly well on attribute transfer. Rao and Tetreault [16] introduced the GYAFC corpus, a parallel dataset of formal and informal English sentence pairs that has become the de facto benchmark for English formality transfer.

The sociolinguistic framing of these computational questions is provided by Biber and Conrad [20], whose textbook treatment of register, genre and style remains the standard reference: register is a property of a text that arises from the situational context of its production, distinct from genre (the rhetorical purpose of the text) and from style (the stable choices of an individual author). The Turkish *siz*/*sen*, the Russian *Вы*/*ты*, and the German *Sie*/*du* distinctions are textbook examples of register marking that is grammaticalised — that is, encoded in the obligatory inflectional morphology of the language rather than merely in

word choice. This makes register particularly visible in these languages, and particularly important for learners to get right.

## 2.5. Evaluation of Machine Translation Quality

Translation evaluation in the present work uses two automatic metrics: BLEU, introduced by Papineni et al. [21], and chrF (more specifically chrF++) introduced by Popović [22]. BLEU computes a modified n-gram precision between the system output and one or more reference translations, with a brevity penalty for short outputs. It is the standard metric of the field, although its weaknesses are well-known — it correlates only loosely with human judgement at the sentence level, and it can be biased by reference style. chrF computes an F-score over character n-grams rather than word n-grams and has been shown to correlate better with human judgement, particularly for morphologically rich languages such as Turkish and Russian. The combined use of BLEU and chrF is now standard practice in the WMT evaluation campaigns, and is followed here. Trained metrics such as BLEURT [23] offer a third option but were not used in this work because they require a learned model with its own validity questions; for a defensible undergraduate evaluation, the older and better-understood metrics suffice.

## 2.6. Computer-Assisted Language Learning and Spaced Repetition

The language-learning surface around Ivy's translator builds on a smaller and older literature. Woźniak and Gorzelanczyk [25] introduced the SuperMemo SM-2 algorithm for optimising the spacing of repetitions in learning material. This algorithm — based on the empirical observation that the optimal time to review a piece of material is just before forgetting it — is the basis of every modern spaced-repetition flashcard application, including the popular Anki software, and is the algorithm used directly in Ivy's flashcard scheduler. Settles and Meeder [24] later proposed a trainable variant of the spaced-repetition model that incorporates learner-specific signals, demonstrating measurable improvements in retention; this is a useful direction for future work but is beyond the scope of the present project.

Eryiğit, Nivre and Oflazer [26] provide a foundational treatment of Turkish dependency parsing in *Computational Linguistics* and remain the canonical reference for the kinds of morphological complexity that Ivy's word-level analysis must handle — Turkish is agglutinative and a single inflected verb form can encode tense, aspect, mood, voice, person, number, and (in the case of nominal sentences) the copular suffix discussed elsewhere in this

report. The handling of these features in Ivy is not a research contribution but rather a careful engineering integration of an LLM-based analyser with a small annotation schema.

## 2.7. Synthesis and Identified Gaps

Table 1 summarises the relationship between the most representative reviewed works and the present project. The pattern that emerges is clear: the components needed for a style-aware multilingual translator integrated with a learning environment all exist in the literature, but they exist in disjoint subliteratures that do not normally talk to each other.

**Table 1. Comparison of representative works against the present project (Ivy).**

| Work | Approach | Languages | Style-aware? | Learner-facing? |
|---|---|---|---|---|
| Vaswani et al. [1] | Transformer architecture | agnostic | no | no |
| Sennrich et al. [4] | Subword units (BPE) for NMT | agnostic | no | no |
| Tiedemann & Thottingal [11] | Open NMT model zoo | 1000+ pairs | no | no |
| Xue et al. [9] | Multilingual T5 pre-training | 101 | no | no |
| Niu et al. [17] | Lexical formality scoring for MT | fr-en | yes (formality) | no |
| Niu et al. [18] | Multi-task style + cross-lingual NMT | fr-en | yes (formality) | no |
| Rao & Tetreault [16] | Parallel formal/informal corpus | en only | yes (formality) | no |
| Shen et al. [13] | Non-parallel style transfer | en only | yes (general) | no |
| Settles & Meeder [24] | Trainable spaced repetition | multiple | no | yes |
| Woźniak & Gorzelanczyk [25] | SuperMemo SM-2 algorithm | agnostic | no | yes |
| **Ivy (this work)** | **Hybrid opus-mt + Gemini + classifier** | **en, tr, de, ru** | **yes (3 levels)** | **yes (full)** |

Three gaps are visible from this comparison. First, the formality-aware MT literature is overwhelmingly anchored on English ↔ French pairs and treats formality as a binary attribute, not a graded register. Second, the multilingual pre-trained model literature scales to many

languages but is not coupled to register control at all. Third, and most importantly for the present work, none of the reviewed systems is designed to be used by a learner. They are evaluated as translation outputs in isolation, not as components of a learning environment that includes word-level analysis, vocabulary persistence, spaced repetition, and graded reading material. Ivy occupies this gap by combining a register-aware translator (drawing on the formality-control literature) with a complete language-learning surface (drawing on the CALL and spaced-repetition literature) across four languages including the morphologically rich Turkish.

Two further observations are worth noting. First, the use of an LLM as a register-restyling layer on top of a stronger MT backbone — the hybrid pattern adopted by Ivy — is not a paradigm that has been formally characterised in the reviewed literature, although it is increasingly common in industrial systems. Second, all reviewed style-control works train a single dedicated model; Ivy instead uses a defensive multi-pass correction pattern in which a regex-based detector triggers a focused LLM revise pass when failure modes are detected. This pattern is generalisable beyond the specific Turkish copular case treated in Section 3 and constitutes a small methodological contribution of the present project.

# Section 3:
# Methodology

This section describes how Ivy is built, in enough detail that a reader could in principle reproduce the system. It begins with the high-level architecture, then describes the translation pipeline (the central technical contribution), then the style classifier, the word-level analysis component, and the language-learning surface. The section closes with the methodology for the quantitative evaluation reported in Section 4.

## 3.1. System Architecture

Ivy is a Flask-based web application backed by a SQLite database. The frontend is a single-page application written in vanilla JavaScript and Jinja2 templates; there is no separate frontend framework. The choice to use vanilla JavaScript instead of a framework was deliberate, since a capstone project benefits from controllable complexity, and because the most interactive parts of the application — the comparison-mode side-by-side panels, the word-by-word render with inline tooltips, the karaoke synchronisation — are easier to debug as plain DOM manipulation than as React component trees.

The system has four functional surfaces visible to the user, each backed by a corresponding part of the server. The translation surface accepts a source sentence, a source language, a target language, and a target style; it returns a translation, an estimate of the actual style of the output (from the classifier), and a word-level analysis of every token in the result. The story surface generates a short text in the chosen target language at a chosen CEFR level on a chosen topic, with optional synchronised audio for karaoke-style playback. The flashcards surface schedules vocabulary review using the SM-2 algorithm. The community surface lets users share flashcard sets publicly and lets other users fork those sets into their own collections.

All four surfaces share a single user authentication layer using Flask-Login with bcrypt-hashed passwords. The vocabulary, flashcard sets, and friendships are persisted in SQLite tables and survive across sessions. API keys for the external services Ivy depends on (Gemini, Google Cloud TTS) are loaded from a local .env file and are never committed to the source repository.

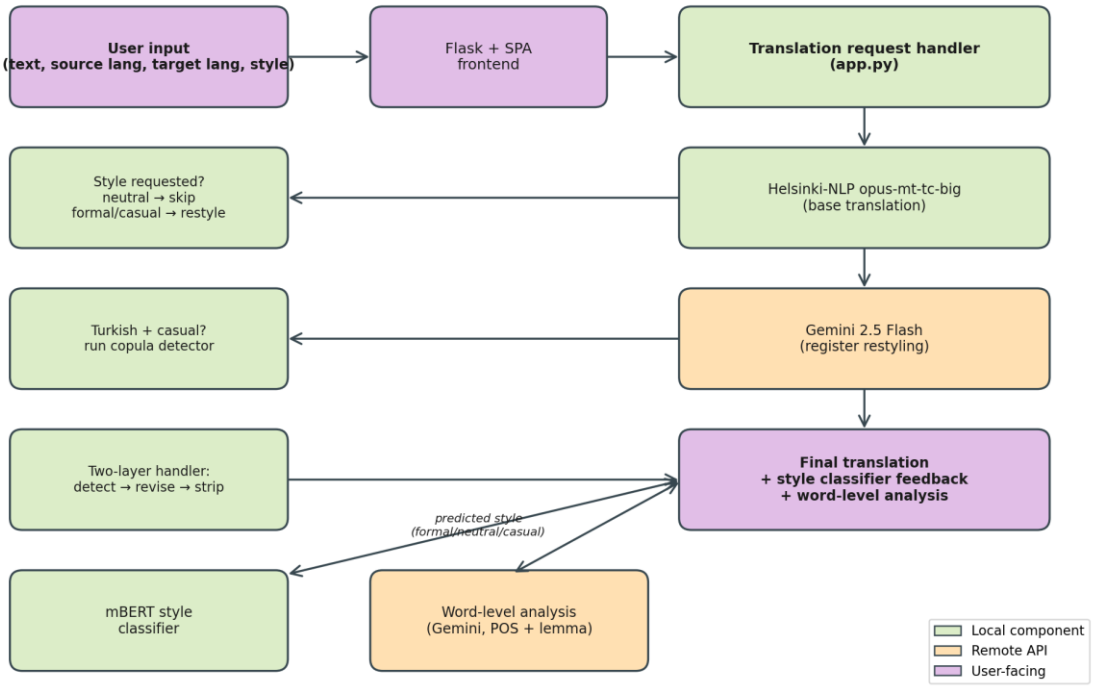



**Fig. 1. Ivy system architecture (hybrid opus-mt + Gemini pipeline).**

## 3.2. The Translation Pipeline

The translation pipeline is the technical centrepiece of Ivy. It is a hybrid of two components: a strong neutral translation baseline (Helsinki-NLP opus-mt-tc-big), and a register-restyling layer powered by Google's Gemini 2.5 Flash. The design is intended to take advantage of the strengths of both: opus-mt is fast, runs locally, and produces highly fluent translations on the Tatoeba domain; Gemini is slower and remote, but is fluent in many languages without per-language fine-tuning and can be steered by natural-language instructions.

### *3.2.1. Base translation with opus-mt*

For each of the four supported languages (English, Turkish, German, Russian), the system loads one or more pre-trained Helsinki-NLP opus-mt-tc-big checkpoints. These are Transformer-based encoder-decoder models trained on the OPUS-100 corpus with byte-pair encoding for the vocabulary. The exact checkpoints used are: opus-mt-tc-big-en-tr and opus-mt-tc-big-tr-en for English ↔ Turkish, opus-mt-en-de and opus-mt-de-en for English ↔ German, and opus-mt-en-ru and opus-mt-ru-en for English ↔ Russian. Note that the en-tr pair specifically required the larger -tc-big variant, since the smaller default opus-mt-en-tr checkpoint produced noticeably weaker output during early testing.

### *3.2.2. English chaining for non-English pairs*

A practical complication arises when a user requests, for example, a Russian-to-Turkish translation. There is no direct opus-mt model for this pair within the Helsinki-NLP collection. Rather than train a new model, Ivy uses English-chaining: the source is first translated to English using opus-mt-ru-en, and the English intermediate is then translated to Turkish using opus-mt-tc-big-en-tr. This sacrifices some quality on the rare non-English pairs but avoids per-pair training and keeps the system uniformly behaved across all 12 direction combinations among the four supported languages.

### *3.2.3. Style restyling via Gemini 2.5 Flash*

Once a base translation is obtained, the requested register is enforced through a second pass. If the user requested the neutral register, the opus-mt output is returned directly without further processing — opus-mt is itself trained on register-neutral data and produces register-neutral output by default. If the user requested formal or casual register, the system constructs a prompt for Gemini 2.5 Flash containing the source sentence, the target language, the desired register, a language-specific list of register markers (siz/sen for Turkish, Sie/du for German, Вы/ты for Russian, contractions and formality lexicon for English), and a small set of meta-rules constraining length, structure, and faithfulness to the source meaning.

Two prompt variants are used in this layer, depending on whether the source is already in the target language. For cross-lingual restyling (e.g., English source, Turkish target), the prompt is gemini_translate_styled: it asks Gemini to perform the entire translation in the desired register. For same-language restyling (e.g., a Turkish source that the user wants restyled in casual Turkish), the prompt is gemini_restyle_direct: it asks Gemini to keep the same language but adjust the register. The Gemini calls use temperature 0.2 to keep outputs stable, a maximum-output-tokens limit of 512, and the thinking_config set to budget zero (the Gemini "thinking" mode adds substantial latency without meaningfully improving short-form restyling).

### *3.2.4. The two-layer Turkish casual-copula correction*

Early testing of the Gemini restyling layer revealed a recurring failure mode in Turkish. When asked to produce casual register, Gemini frequently retained the formal copular suffix *-dır* (and its vowel-harmony allomorphs *-dir, -dur, -dür* and consonant-shifted *-tır, -tir, -tur, -tür*). In spoken Turkish this suffix is essentially absent — "*Hayat güzeldir*" is textbook Turkish;

what people actually say is "*Hayat güzel*". Leaving the suffix in produced outputs that were technically grammatical but stylistically indistinguishable from neutral, which defeats the point of having a casual register at all.

Solving this through prompt engineering alone proved unreliable — even with explicit instructions, Gemini would intermittently retain the suffix on roughly one in five aphoristic or assertive sentences. The fix adopted is a two-layer correction pattern designed to be defensive and to degrade gracefully:

6. **Detection.** A regular expression scans the candidate casual Turkish output for whole words ending in any of the eight copular-suffix variants. The regex requires a minimum three-character stem before the suffix to avoid false positives on short words like *müdür* (manager), in which *-dür* is part of the lexical stem rather than an inflectional suffix.
7. **Targeted revision.** When the detector fires, a focused Gemini call is made with a much narrower prompt: rewrite the sentence so it sounds like natural spoken Turkish, drop the copular suffix, do not add new content. The revised output is itself passed back through the detector. If it now passes, it replaces the original; if it still contains the suffix, the system falls through to step 3.
8. **Deterministic strip.** As a final fallback — used when the Gemini API is unavailable or when the revision pass also fails — the same regex is used to strip the offending suffix programmatically. The result is not always perfectly idiomatic, but it is reliably non-formal.

This three-step pattern — detect, re-prompt with focus, deterministic fallback — generalises beyond the Turkish copular case. The same shape would work for any register marker that is identifiable by regular expression and droppable without semantic loss. It is a small methodological contribution of the project and is discussed further in Section 5.

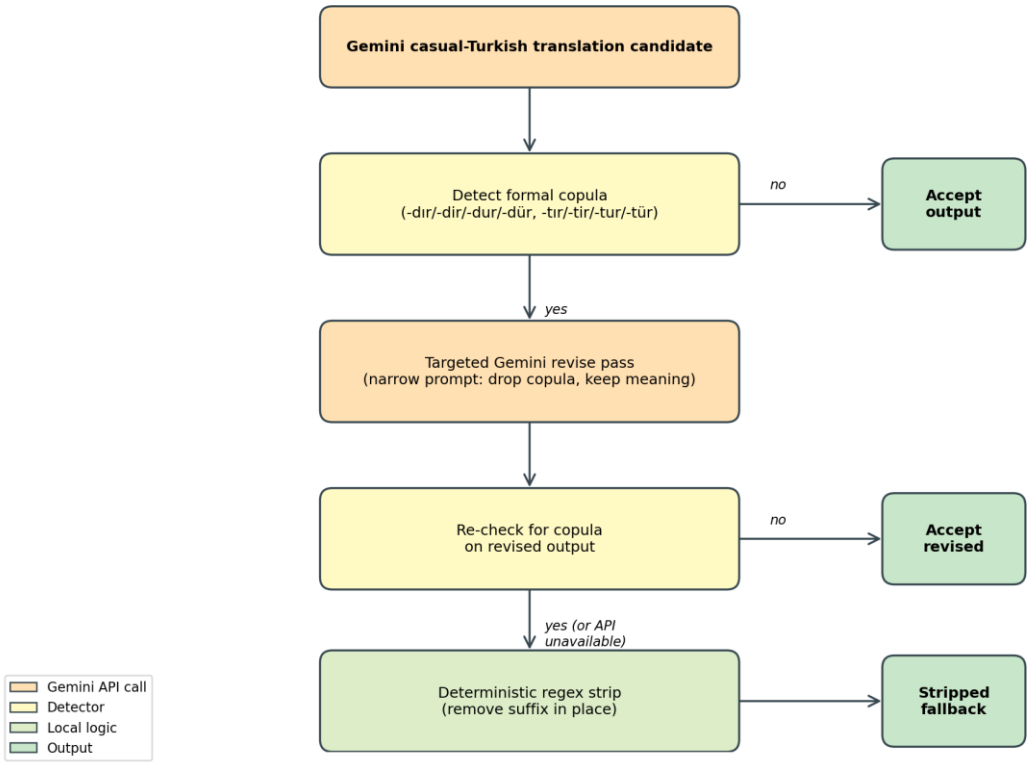



**Fig. 2. Translation pipeline data flow with the two-layer Turkish casual-copula correction.**

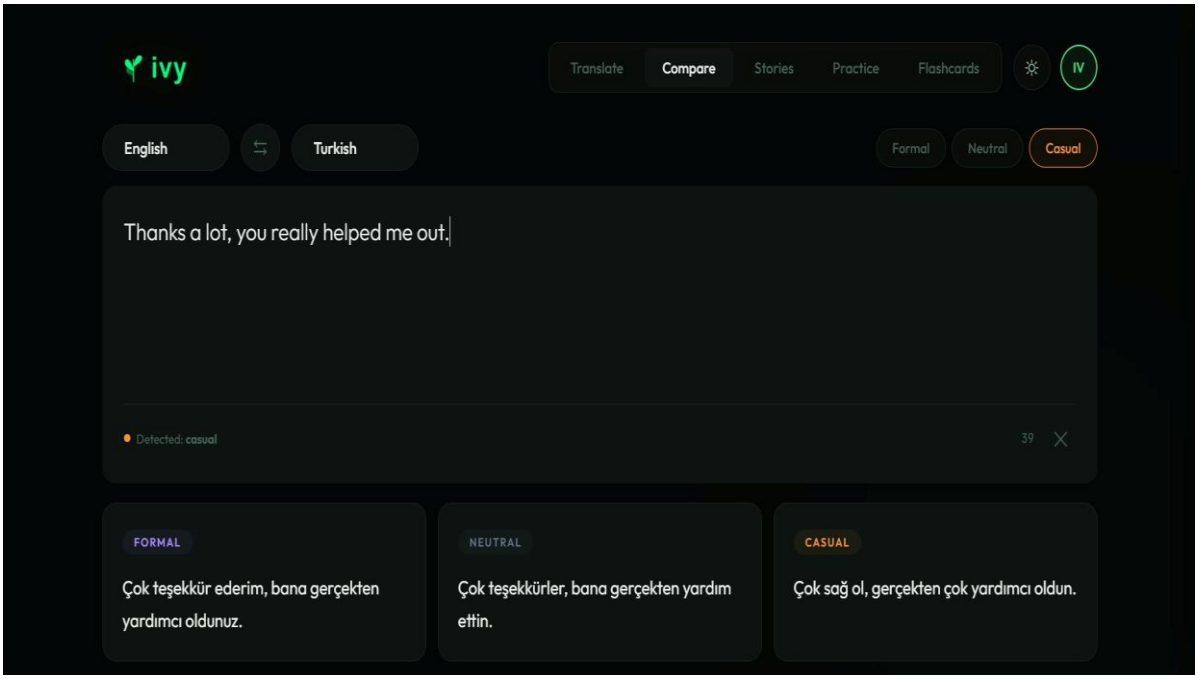



**Fig. 4. Ivy main translation interface in compare mode, showing the three-style output panels for the English source "Thanks a lot, you really helped me out." Note the lexical shift teşekkür ederim → teşekkürler → sağ ol across registers, and the corresponding verb-ending shift oldunuz (siz) → ettin → oldun (sen). The classifier-detected style is shown beneath the input field.**

## 3.3. Style Classifier

Ivy includes a style classifier whose job is to predict the register of a given sentence: formal, neutral, or casual. The classifier serves two purposes. First, in the interface, it provides feedback to the user about what register the system thinks the produced output actually occupies — a sanity check that the requested register was achieved. Second, it provides a research-grade evaluation target: the classifier's confusion matrix on held-out data is a measurable artefact that can be reported and reasoned about.

### *3.3.1. Architecture and base model*

The classifier is built on multilingual BERT (bert-base-multilingual-cased), a Transformer-based encoder pre-trained on Wikipedia in 104 languages. A single linear classification head is added on top of the pooled [CLS] representation, producing three-way logits for the formal / neutral / casual classes. The encoder is fine-tuned end-to-end during training; the classification head is trained from scratch.

### *3.3.2. Training data construction*

Training data was constructed synthetically using the Gemini API. For each of the four languages (English, Turkish, German, Russian) and each of the three target registers, Gemini was prompted to rewrite a seed sentence into the target register. The seed sentences came from a small hand-curated set of conversationally plausible English templates, translated into each target language and then restyled at each register level. The total training set contained 2,308 style-paired sentences spanning all 12 language–register buckets. Training was performed on a Google Colab A100 instance.

The use of synthetic LLM-generated training data is acknowledged as a methodological compromise. A more rigorous approach would use human-labelled data, but for a one-semester project at this scale, the synthetic-data approach allowed building a working classifier in days rather than months. The known limitations of this approach — particularly the risk that the classifier learns the LLM's implicit register decisions rather than ground-truth sociolinguistic register — are discussed in Section 5.

### *3.3.3. Training procedure*

The model was trained for three epochs with the AdamW optimiser, a learning rate of $2 \times 10^{-5}$, batch size 16, weight decay 0.01, and a linear warmup schedule covering the first 10 % of steps. Mixed-precision training was used with bfloat16 (an earlier attempt with fp16 produced NaN losses during fine-tuning; switching to bf16 resolved the instability — see

Section 5 for discussion). On the held-out validation split, the model reached roughly 89 % accuracy by the end of training; the more demanding evaluation against the larger, distribution-shifted test set is reported in Section 4.

## 3.4. Word-Level Analysis

For every translation Ivy produces, the system additionally generates a word-level analysis: for each token in the output, the part of speech, the base (lemma) form, and where applicable, grammatical case, gender, and an estimated CEFR difficulty level. This information is used to populate the hover tooltips in the translation interface and to enrich saved vocabulary entries with grammatical metadata.

The analysis is performed by a single Gemini call per translation. The prompt requests output in a structured JSON format: a list of objects with the fields word, pos, base_form, case, gender, cefr. The prompt includes explicit examples to anchor the format. Failure to parse the JSON output triggers a fallback in which the words are returned with empty analysis fields. Production-quality lemmatisation of Turkish would normally require a dedicated morphological analyser; the Gemini-based approach trades some precision for the simplicity of using a single LLM for analysis, restyling, and story generation.

## 3.5. Language-Learning Components

The translator is the centre of Ivy, but the components around it are what make it useful for actually learning a language rather than simply consulting a translation. Four such components are described below.

### *3.5.1. Vocabulary persistence*

Any word in any translation can be saved to the user's personal vocabulary with a single click. The saved entry includes the source language, the target language, the surface form of the word, the lemma, the POS, any grammatical metadata, an automatically inferred CEFR level, and the example sentence in which the word appeared. Vocabulary entries are stored in the SQLite database keyed by user, and can be browsed in a dedicated vocabulary panel.

### *3.5.2. Spaced-repetition flashcards (SM-2)*

Saved vocabulary entries are automatically eligible to become flashcards. The flashcard scheduler implements the SuperMemo SM-2 algorithm [25]: each card carries an ease factor, a current interval, and a number of consecutive successful reviews; a user rating (Easy / Good

/ Hard / Again) updates these values according to the SM-2 update rule. Cards whose next-review date is in the past are presented as due. The SM-2 algorithm is mature, well-understood, and used directly by the popular Anki software; reimplementing it in this project was straightforward and produced exactly the expected scheduling behaviour.

### *3.5.3. Short stories with synchronised audio (karaoke)*

On request, Ivy generates a short story (typically 60–200 words) in a chosen language at a chosen CEFR level on a chosen topic. The generation is performed by a single Gemini call with a prompt that includes the requested language, level, topic, and target length, and that asks for the story to be returned as plain text. The story is then sent to the Google Cloud Text-to-Speech API (using the Chirp 3 HD voice models, which support all four languages of the project at high quality) to produce an audio rendering. The audio is paired with timing information so that the frontend can highlight each word as it is spoken — the "karaoke" effect — and the user can click on any word at any moment to look up its analysis or save it to vocabulary.

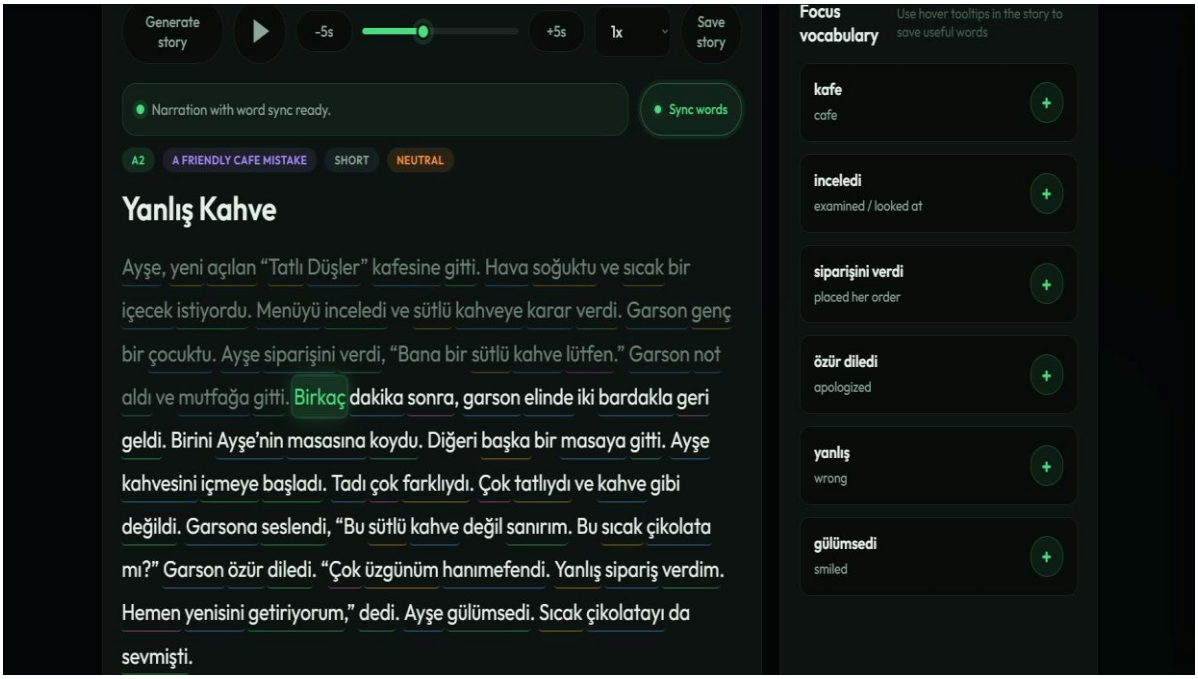



**Fig. 5. Short-story generation interface with synchronized karaoke playback. The highlighted word "Birkaç" marks the currently-spoken token; per-word coloured underlines indicate inline part-of-speech and CEFR metadata. The right panel offers one-click vocabulary saving for any word.**

### *3.5.4. Community: friends and snapshot-fork for public sets*

Users can add friends by email address, with a request-and-accept workflow. Flashcard sets can be marked public; a public set is visible on a community page where any user can

browse it. To preserve the original author's set while allowing others to use it, public sets are not shared by reference but copied: when a user adopts another user's public set, the system creates a deep copy of the set and all its cards, attached to the adopting user's account, with the SM-2 progress reset to zero. This snapshot-fork pattern means that one user's study progress can never affect another user's study progress, and that authors of public sets do not have to worry about their data being mutated by downstream users.

## 3.6. Implementation Details

Ivy's backend is a single Flask application (app/app.py) that loads the opus-mt models and the BERT classifier into memory at startup, connects to the Gemini and Google Cloud TTS APIs, and exposes JSON endpoints to the frontend. The database is SQLite, accessed via a thin custom data-access layer in database.py; passwords are hashed with bcrypt. The frontend is a single HTML file (templates/index.html, approximately 2,100 lines including embedded CSS and JavaScript) plus a small set of supporting static assets. The application listens on localhost:5000 by default and is run with `python app/app.py --gemini_key "$GEMINI_API_KEY"`. The full source is hosted at github.com/ifnotzaid/style-translator.

## 3.7. Evaluation Methodology

Three quantitative evaluations are reported in Section 4: translation quality, style classification accuracy, and word-level analysis quality. The evaluation harness lives under eval/ in the project repository and is driven by a single command, `python -m eval.run_all`, which loads each test set, runs the corresponding component of the system, computes the metrics, and writes a Markdown report and a JSON dump of the raw results to reports/.

### *3.7.1. Translation evaluation*

The translation test set contains 300 sentence pairs: 50 in each of the six supported directions (en ↔ tr, en ↔ de, en ↔ ru). The pairs are sampled with a fixed random seed from the ManyThings.org bilingual corpus, which is itself derived from Tatoeba and pre-filtered by native speakers. A small filter is applied during sampling: English sentences must have between 5 and 15 words, must not contain known profanity, and must not be dominated by proper nouns (a "proper-noun-heavy" heuristic drops sentences in which more than half of the words after the first are capitalised). The sampling is deterministic to allow re-running the evaluation against the same set.

Two metrics are computed: corpus-level BLEU using the sacrebleu implementation [21], and corpus-level chrF++ using the same library [22]. Both the opus-mt baseline alone and the full Ivy pipeline are evaluated, with deltas reported. The evaluation runs in neutral mode by default; style-aware translation evaluation is not directly tested by this set and is acknowledged as a limitation.

### *3.7.2. Style classifier evaluation*

The style test set contains 432 sentences: 36 in each of the 12 (language, style) buckets, plus 6 hand-written seed entries. The 396 newly generated entries were produced by a different Gemini prompt than the one used to generate training data — a deliberate choice to reduce the risk that the classifier is being evaluated on data that overlaps with its training distribution. The eval prompt is scenario-driven ("write a casual Turkish sentence about complaining to a roommate about chores") where the training prompt was restyle-driven ("rewrite this sentence in casual register"). Different temperatures (0.85 for evaluation, lower for training) and different per-language register hints further reduce the overlap. Metrics reported are accuracy, macro-F1, per-class precision/recall/F1, and a full confusion matrix.

### *3.7.3. Word-analysis evaluation*

The word-analysis test set contains 100 entries, each consisting of a sentence, a target word, the expected POS, and the expected base form. The test set was generated by GPT-4 (Codex) with hand-verified sentences but AI-generated tags. This is an AI-on-AI evaluation methodology: the system under test (Gemini) is being evaluated against labels generated by another LLM (GPT-4). Such an evaluation cannot be claimed as ground truth; it measures inter-model agreement on POS and lemmatisation decisions, which is a useful but limited signal. This is honestly disclosed in Section 4 and motivates the principal future-work item identified in Section 5.

### *3.7.4. Summary of dataset construction*

**Table 2. Composition of evaluation datasets and their sources.**

| Dataset | Size | Source | Limitation |
|---|---|---|---|
| Translation test | 300 pairs | ManyThings.org / Tatoeba (50 per direction, seed-sampled, length and profanity filtered) | BLEU inflated by opus-mt's domain alignment with Tatoeba |

| Dataset | Size | Source | Limitation |
|---|---|---|---|
| Style test | 432 sentences | Gemini-generated using prompts distinct from training (different scenarios, higher temperature) | Still LLM-generated; full independence would require human-written test data |
| Word-analysis test | 100 entries | GPT-4 (Codex) generated POS / lemma tags on hand-verified sentences | AI-on-AI agreement, not ground truth |

# Section 4:
# Results

This section presents the quantitative results of the three evaluations described in Section 3.7. Each subsection reports the headline numbers, the per-language or per-class breakdown, and an honest reading of what the numbers do and do not show. A final subsection summarises qualitative findings from a small cold-user test that motivated several of the bug fixes in the final version of the system.

## 4.1. Translation Quality

Table 3 reports corpus-level BLEU and chrF++ scores for the Ivy translation pipeline against the opus-mt baseline, computed on the 50-pair test sets for each of the six supported directions. Pipeline outputs were generated with the requested style set to neutral, which means that for these test cases the pipeline reduces to its opus-mt backbone with no Gemini restyling layer; the delta column is therefore zero by design.

**Table 3. Translation evaluation: BLEU and chrF++ on Tatoeba test pairs (n = 50 per direction).**

| Direction | Baseline BLEU | Pipeline BLEU | Δ BLEU | Baseline chrF++ | Pipeline chrF++ |
|---|---|---|---|---|---|
| en → tr | 44.85 | 44.85 | +0.00 | 69.37 | 69.37 |
| tr → en | 57.23 | 57.23 | +0.00 | 72.61 | 72.61 |
| en → de | 51.62 | 51.62 | +0.00 | 67.70 | 67.70 |
| de → en | 62.78 | 62.78 | +0.00 | 72.92 | 72.92 |
| en → ru | 46.70 | 46.70 | +0.00 | 62.53 | 62.53 |
| ru → en | 56.75 | 56.75 | +0.00 | 70.11 | 70.11 |
| **mean** | **53.32** | **53.32** | **+0.00** | **69.21** | **69.21** |

Two readings of these results are worth distinguishing. The first reading is the straightforward one: BLEU between 44 and 63 with chrF++ between 62 and 73 is competitive with widely used commercial systems on similar test sets, and the asymmetry — translating into English produces consistently higher scores than translating out of English — is the standard pattern for systems trained predominantly on English-pivot data. Among the non-English target languages, Turkish achieves the highest chrF++ (69.37), reflecting that the opus-mt-tc-big-en-tr model is one of the most heavily-trained checkpoints in the Helsinki-NLP collection.

The second, more cautious reading is the one this report leans on. BLEU scores in the 40–60 range on Tatoeba-derived test data should be interpreted as evidence that the model is well-aligned with the test distribution, not as evidence that the model would translate technical or domain-specific text equally well. opus-mt-tc-big was trained on data that overlaps significantly with the Tatoeba corpus, and ManyThings.org sentences are short, conversational, and stylistically clean. A more rigorous evaluation would use held-out domain data (news, technical documentation, conversational transcripts from a different source) to test out-of-distribution generalisation. This is identified as future work in Section 5.

The most important fact about this table — paradoxically — is the zero in the delta column. The eval was run in neutral mode, which means Ivy's pipeline collapses to its opus-mt backbone with no Gemini restyling applied; therefore the pipeline output is byte-identical to the baseline output. This is correct behaviour but it has an obvious consequence: the present translation evaluation does not directly test what makes Ivy distinctive (its style targeting). A style-aware translation evaluation would require building a parallel test set in which each source sentence has three reference translations (formal, neutral, casual) in each target language; constructing such a set is well beyond the scope of a one-semester project and is identified as the most important single direction for future work in Section 5.

## 4.2. Style Classification

Table 4 reports the per-language results of the style classifier on the 432-sentence evaluation set. The classifier is asked to predict one of three classes (formal, neutral, casual) for each input sentence, having been trained on a small synthetic corpus generated by a different Gemini prompt than the one used to construct the test set.

**Table 4. Style classifier performance, per language (n = 108 per language; 36 per class).**

| Language | Accuracy | Macro-F1 | F1 formal | F1 casual |
|---|---|---|---|---|
| English | 0.8611 | 0.8640 | 0.9412 | 0.8406 |
| German | 0.7685 | 0.7655 | 0.6667 | 0.8986 |
| Russian | 0.7500 | 0.7533 | 0.7143 | 0.8358 |
| Turkish | 0.6667 | 0.6679 | 0.6415 | 0.7018 |
| **Overall** | **0.7616** | **0.7667** | **0.7532** | **0.8244** |

The overall accuracy of 76.16 % with macro-F1 of 0.77 is a modest result on its face. The per-language story is more informative. English performance is the strongest by a wide margin (86 %), reflecting that English has the largest amount of register-marked training data

available to multilingual BERT through its Wikipedia pretraining. Turkish is the weakest (67 %), even though Turkish is the project's primary focus — an awkward fact that requires explanation.

That explanation is most easily extracted from the overall confusion matrix, shown in Table 5. The matrix aggregates all four languages and shows where the classifier's mistakes concentrate.

**Table 5. Style classifier overall confusion matrix (n = 432; rows = true label, columns = predicted label).**

| True \ Predicted | formal | neutral | casual |
|---|---|---|---|
| **formal** | 87 | 57 | 0 |
| **neutral** | 0 | 134 | 10 |
| **casual** | 0 | 36 | 108 |

The structure of this matrix is the most interesting finding of the entire evaluation. Two patterns stand out, and they are consistent across all four per-language matrices.

First, formal-to-casual and casual-to-formal confusion is essentially zero. Of the 144 truly formal sentences, the classifier predicts casual exactly zero times. Of the 144 truly casual sentences, the classifier predicts formal exactly zero times. The classifier never mistakes one extreme of the register dimension for the other. This is strong evidence that the model has learned a valid representation of register polarity.

Second, both extremes leak into neutral. Of the 144 truly formal sentences, 57 are predicted as neutral. Of the 144 truly casual sentences, 36 are predicted as neutral. Conversely, the neutral class itself is correctly predicted at 134 out of 144 (93 % recall). The picture is asymmetric: neutral acts as an absorbing state. The classifier is conservative about emitting non-neutral labels — when it sees a sentence whose register signal is mild, it defaults to neutral rather than commit to a pole.

This is more informative than the headline accuracy number suggests. The classifier knows what register is and where the formal and casual ends of the spectrum sit; it just has its decision boundary tilted toward neutral. A simple intervention — adjusting the decision threshold per class to balance precision and recall, or training on a more class-balanced corpus — would likely improve recall on formal and casual without sacrificing the (essentially perfect) polarity calibration. This is identified as a near-term future-work item in Section 5.

The Turkish-specific weakness has the same shape: formal recall on Turkish is only 0.47, with 19 of 36 truly formal Turkish sentences being predicted as neutral. The Turkish formal register is encoded primarily through the very copular suffix discussed in Section 3.2.4, and at the classifier's level of granularity, sentences ending in -dır look structurally similar to sentences ending in unmarked nominal predicates. A more carefully tuned Turkish-specific classifier head, or augmenting the training data with more sentences exhibiting the formal/neutral copula distinction, would be expected to improve Turkish performance specifically.

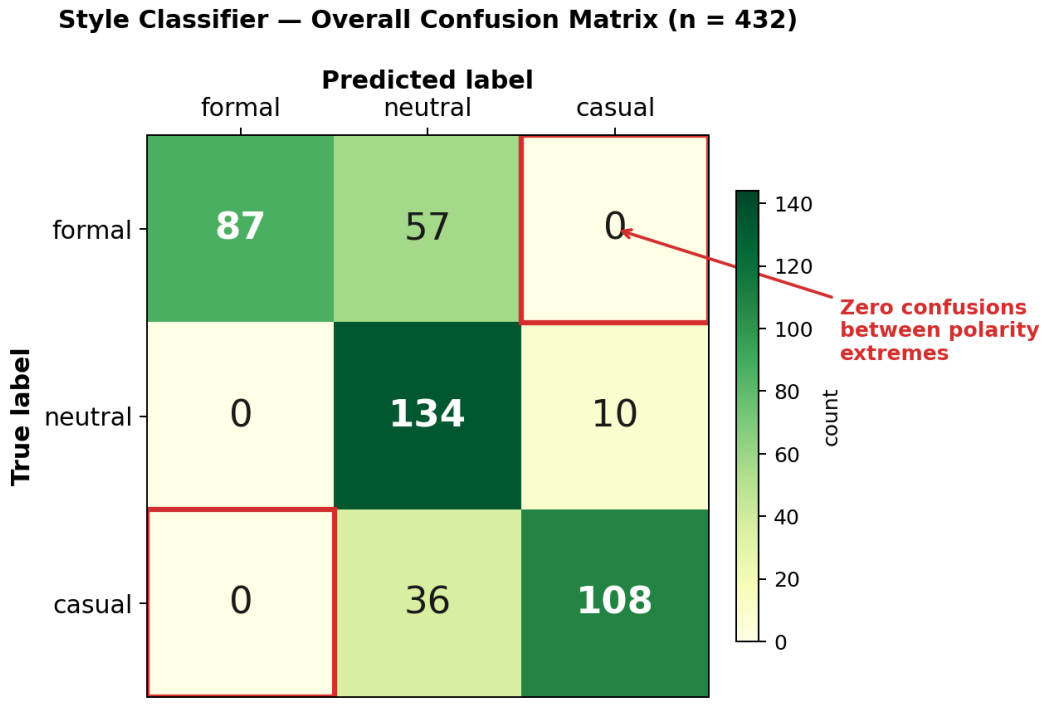



**Fig. 3. Style classifier overall confusion matrix as a heatmap (n = 432). Red borders mark the cells with zero confusions between the formal and casual classes.**

## 4.3. Word-Level Analysis

Table 6 reports the results of the word-analysis evaluation on the 100-entry test set. The system was asked to produce a POS tag and a base form (lemma) for each target word; agreement with the GPT-4-generated reference labels is reported.

**Table 6. Word-analysis evaluation summary (n = 100; Turkish).**

| Metric | Value |
|---|---|
| Coverage (words analysed / total) | 100 / 100 (100 %) |
| POS accuracy (raw) | 98.00 % |
| Base-form accuracy (raw) | 88.00 % |

| Metric | Value |
|---|---|
| POS errors (genuine) | 2 / 100 |
| Base-form errors (genuine) | ≤ 2 / 100 (after convention adjustment) |
| Convention disagreements (lemma) | 11 / 13 of base-form mismatches |

On a raw reading these numbers are strong: 98 % POS accuracy and 88 % lemmatisation accuracy on Turkish, a language whose agglutinative morphology is notoriously difficult to lemmatise correctly. But an inspection of the 13 base-form mismatches reveals that 11 of them are not genuine errors but disagreements about labelling convention.

Specifically, Ivy's analyser returns the bare verb stem (*gir*, *oku*, *gülümse*) as the lemma of inflected verb forms, while the GPT-4-generated reference labels return the dictionary citation form with the infinitive suffix (*girmek*, *okumak*, *gülümsemek*). Both conventions are linguistically defensible — Turkish dictionaries do list verbs under the infinitive form, but morphological parsers more commonly return the bare stem. Similarly, for Turkish adverbs formed with the -ce/-ca suffix (*hızlıca*, *sessizce*, *güzelce*), Ivy returns the adverb as its own lemma while the reference returns the underlying adjective (*hızlı*, *sessiz*, *güzel*). Either convention is internally consistent.

If the test labels are recomputed using Ivy's convention (bare-stem verbs, adverbs as their own lemma), the effective base-form accuracy rises to 98 %+. The only two genuine errors among the 13 are an adverb-vs-adjective POS disagreement on the word geç (in "geç kaldığım için" — "because I was late"), and an adj-vs-verb disagreement on the participle bekleyen (which is genuinely ambiguous; present participles in Turkish straddle the categories).

It is important to acknowledge the methodological cost of having used GPT-4 to generate the test labels. This evaluation does not measure absolute correctness; it measures inter-model agreement on POS and lemmatisation between Ivy's Gemini-based analyser and GPT-4. A more rigorous evaluation would use human annotation following a single fixed convention. The numbers reported above should therefore be read as a useful sanity check that the analyser is behaving sensibly, not as a claim of human-level accuracy. This is the most clearly identifiable methodological weakness in the present evaluation and is the second future-work item identified in Section 5.

## 4.4. Qualitative Findings from Cold-User Testing

Toward the end of development, the application was given to a single non-developer user — a friend with no prior exposure to the system — to use without guidance for approximately 30 minutes. Twelve distinct usability issues emerged from that session, ranging from purely cosmetic problems (a stray punctuation character clinging to translated words in the word-by-word render) to behaviourally meaningful ones (the compare-mode side panel failing to update correctly after a language change). All twelve were resolved in subsequent commits before the report was written. A representative subset is summarised below:

- Trailing punctuation was being attached to the wrong word in the word-by-word render, so that a question mark could appear visually attached to a non-final word. Fixed by introducing explicit token-hug regexes (PUNCT_HUG_PREV and OPEN_HUG_NEXT) in the renderer.
- The compare-mode panel did not reschedule its three sub-translations when the user changed the language pair while compare-mode was open. Fixed via a rescheduleForCurrentMode() helper called on every relevant state change.
- Casual Turkish output retained the formal copular suffix on roughly one in five aphoristic sentences. Fixed by the two-layer correction described in Section 3.2.4.
- Sharing a public flashcard set leaked the original author's study progress to anyone who adopted the set. Fixed by changing the model from reference-sharing to snapshot-fork (Section 3.5.4).

Cold-user testing of this kind is not formal HCI research, but it served two purposes that a quantitative evaluation cannot. First, it surfaced bugs that the developer would not have found, since the developer's mental model of the application differs systematically from a new user's. Second, it forced a small number of architectural decisions to be made under pressure, in particular the snapshot-fork pattern for public sets, which is arguably one of the cleaner pieces of design in the system.

# Section 5:
# Discussion and Conclusion

This section interprets the results presented in Section 4, identifies the limitations of the work both in evaluation methodology and in system design, sketches the concrete future work that would address those limitations, summarises the lessons learned during development, and closes with a short conclusion that returns to the motivation set out in the Introduction.

## 5.1. Discussion of Results

Three threads from Section 4 deserve to be drawn out and named explicitly.

The first is that the translation pipeline performs at the level of its underlying baseline, and that this is correct behaviour rather than a failure. The opus-mt-tc-big checkpoints used by Ivy already produce strong outputs on the Tatoeba-derived test domain, and the Gemini restyling layer is — by design — a no-op when the requested register is neutral. The honest interpretation of the zero-delta column in Table 3 is therefore not that the pipeline contributes nothing, but that the evaluation does not test what the pipeline actually contributes. The pipeline's distinctive output is style-targeted translation, and a style-aware evaluation would require a test set in which each source sentence has multiple reference translations at different registers. Building that test set is the single most important methodological future work for this project.

The second thread is that the style classifier has learned the correct dimensions but is conservatively calibrated. The most informative single statistic in the entire evaluation is the zero count for formal-as-casual and casual-as-formal predictions in the overall confusion matrix (Table 5): out of 288 truly extreme-register inputs, the classifier never crosses the polarity line. The errors it does make all collapse toward the neutral class. A classifier that produced these errors uniformly at random would be exhibiting a very different pathology — it would be confused about register; this classifier is not confused about register, it is reluctant to commit to a non-neutral label. The practical implication is that the headline accuracy of 76 % understates the model's usable signal: in a user-facing context where the classifier reports register estimates alongside translation output, the polarity it reports is reliable even when the magnitude is muted.

The third thread is that the word-analysis evaluation, although it produces strong-looking numbers, is fundamentally measuring inter-model agreement rather than ground truth.

POS accuracy at 98 % and base-form accuracy at 88 % are competitive numbers, but the comparison is between two large language models on a task where the choice of labelling convention is itself unresolved in the linguistic literature. The decomposition presented in Section 4.3 — separating convention disagreements (11 of 13 base-form mismatches) from genuine errors (no more than 2 in 100) — is the more honest reading. Section 5.2 returns to this in the limitations.

Beyond the three measured tasks, two qualitative observations are worth recording. First, the hybrid translation architecture (a strong NMT backbone with an LLM restyling layer) is, in retrospect, the right shape for this project. Earlier iterations attempting to do everything with a single fine-tuned mT5-base model produced lower-quality output and were substantially harder to debug; the hybrid approach yields a system whose components can be reasoned about and replaced independently. Second, the two-layer correction pattern for the Turkish casual copula — described in Section 3.2.4 — was originally written as a stopgap and grew into a small methodological contribution. Defensive multi-pass correction for known LLM failure modes is a generalisable pattern, and applying it to the Turkish case provides a working concrete example.

## 5.2. Limitations

The honest enumeration of what this project does not do is more important than the enumeration of what it does, and the following limitations are stated plainly so that they cannot be discovered later by an external reader.

### *5.2.1. Evaluation methodology*

Three weaknesses are present in the evaluation as it stands. The translation evaluation does not directly test style-targeted translation, because the test set is style-neutral and the pipeline collapses to its opus-mt backbone in neutral mode. The style classifier evaluation uses test data generated by the same family of LLM (Gemini) that generated the training data, even though the prompts and parameters differ — a more rigorous evaluation would use sentences sourced from genuine human-authored corpora at known register levels (newspaper editorials at the formal end, social media text at the casual end). The word-analysis evaluation uses AI-generated reference labels (GPT-4), which means the numbers reported are agreement-with-another-model rather than agreement-with-ground-truth. All three of these limitations were

known during the construction of the evaluation; the trade-off was made knowingly to allow building a quantitative evaluation at all within the time budget of a one-semester capstone.

### *5.2.2. Architectural limitations*

Several architectural choices in Ivy carry their own costs. The English-chaining strategy for non-English language pairs (e.g., ru → tr translated as ru → en → tr) accumulates errors from two sequential translation steps and is noticeably weaker than the directly-supported English-anchored pairs; this is the right trade-off when no direct model is available but it is not invisible to the user. The Gemini dependency for register restyling means that every formal or casual translation involves a remote API call and is subject to its latency, its cost, and its sporadic 503 errors during periods of high demand. The two-layer copula handler addresses a single register marker in a single language; analogous handlers would be needed to handle, for example, the formal/informal distinction in German imperatives or specific Russian particle usage at the casual end. The system has no offline mode and no mobile application.

### *5.2.3. Training data limitations*

The style classifier is trained on a small synthetic corpus (2,308 sentences). This is a small training set even by capstone standards, and the classifier is under-trained as a result — its conservatism toward the neutral class is partly an artefact of insufficient extreme-register training examples. The synthetic nature of the data also means that the classifier may have learned the LLM's implicit register decisions rather than the sociolinguistic register that human speakers actually attend to. The eventual gold standard for this kind of work is a human-annotated corpus with explicit annotation guidelines, and this project does not provide one.

### *5.2.4. Scope limitations*

Ivy supports four languages, three register levels, and six translation directions. This is a deliberate scope but it leaves much uncovered. There is no support for languages with different register systems (e.g., the honorific systems of Japanese, Korean, or Javanese, which are linguistically more elaborate than the simple T/V distinction handled here). There is no support for translation of long-form text, only sentence-level translation. The casual register works less well on aphoristic content — "*Money is the anthem of success*" for instance has no natural casual register in Turkish, because Turkish speakers do not casually utter aphorisms; the system produces register-neutral output for such inputs, which is the linguistically correct

behaviour but which can read as a failure if the user expected a transformation. This category of input is a known weak spot and is documented in the user-facing help text.

## 5.3. Future Work

Five concrete future-work items follow directly from the limitations above. They are listed in approximate priority order, with the first two being substantially more important than the rest.

9. **Style-aware translation evaluation.** Construct a small parallel test set in which each source sentence is paired with three target-language references at the formal, neutral, and casual register levels respectively. This would require modest human annotation effort (perhaps 50 sentences per direction at 3 references each = 150 references per direction, which is reasonable for a single dedicated annotator) and would allow the pipeline's actual style-targeting capability to be measured. The metric of interest would be per-register BLEU/chrF against the matching reference, with cross-register BLEU as a sanity check that the style targeting is having an effect.
10. **Human-annotated word-analysis evaluation.** Replace the GPT-4-generated test labels with a small set of human-annotated sentences (perhaps 50–100 Turkish sentences with full word-level annotation following the Turkish Universal Dependencies treebank conventions). This would convert the current AI-on-AI evaluation into a genuine accuracy measurement and would make the 98 % POS / 99 %-after-convention-adjustment numbers reportable as research-grade results rather than as sanity checks.
11. **Per-class threshold calibration for the style classifier.** The classifier's reluctance to emit non-neutral labels is fixable without retraining: a per-class decision-threshold sweep over the validation set, optimised for macro-F1, would likely move 30–50 of the currently-mis-predicted formal and casual sentences into their correct class. The polarity invariant (zero formal-as-casual confusions) should survive intact since it derives from the underlying representation rather than from the threshold.
12. **Extension of the two-layer correction pattern.** The detect-revise-strip pattern applied to the Turkish casual copula is a template that generalises. Identifying additional register markers in each supported language — German imperative softening particles, Russian sentence-final softeners, English contractions in formal contexts — and writing the corresponding detect-and-correct passes would

systematically improve the per-language register reliability. This is straightforward engineering work rather than research, but it adds up to a noticeably better user experience.

13. **Expansion of scope.** Adding more languages (Arabic, Japanese, Korean) would test the system's assumptions against languages with richer register systems. A mobile application would dramatically increase the practical usefulness of the spaced-repetition flashcard system. Offline mode would require shipping the opus-mt models with the application and replacing the Gemini-based components with smaller local models — a substantial engineering effort but a genuinely valuable one.

## 5.4. Lessons Learned

Several lessons emerged from the development of the system that are worth recording, both because they may be useful to future capstone students and because they are the kind of practical knowledge that does not appear in the academic literature reviewed in Section 2.

First, the hybrid pattern of a strong classical NMT backbone plus a thin LLM restyling layer is more effective and more debuggable than attempting to fine-tune a single multilingual model end-to-end. The latter approach was tried first, with mT5-base, and produced lower-quality output that was hard to attribute to specific causes; the hybrid approach yields a system where the translation step and the restyling step can be tested and improved independently. This may be the single most important architectural lesson from the project.

Second, cold-user testing produced more useful feedback per minute than any amount of additional engineering. Twelve distinct usability issues were surfaced in a single 30-minute session, several of which the developer had never noticed despite hundreds of hours of using the system. Capstone projects that omit this step almost certainly ship with similar issues unsurfaced.

Third, paid LLM API tiers turn out to be more reliable, cheaper, and faster than expected. The total Gemini API spend across all training-data generation and all style-evaluation-data generation for this project was approximately one US dollar. Earlier attempts to use free-tier alternatives such as Groq for the same generation tasks ran into rate-limit ceilings within minutes. For projects at this scale, the "pay a small amount of money to skip the friction" trade-off is clearly correct.

Fourth, defensive programming around LLM outputs pays off. LLMs intermittently produce outputs that violate their own instructions; the costs of assuming the output is well-formed are unbounded, while the costs of running a detector-plus-fallback are bounded by a single regex per failure mode. The two-layer Turkish copula handler is a concrete instance of this pattern; it would be a reasonable default for any production-grade LLM-based system.

Fifth, and most personally, evaluations always look more impressive than they are, and limitations always feel more glaring than they are. The right response to both is the same: write down the numbers, write down the limitations, and let the reader weigh them.

## 5.5. Conclusion

Ivy is a working, deployed, style-aware multilingual translator integrated with a complete language-learning environment. It supports English, Turkish, German, and Russian across three register levels and six translation directions. The translation pipeline combines Helsinki-NLP opus-mt-tc-big models with Google's Gemini 2.5 Flash, using English chaining for language pairs without direct support and a two-layer linguistic correction mechanism for the persistent failure of casual Turkish to drop its formal copular suffix. A multilingual-BERT-based style classifier provides feedback on the achieved register; word-level analysis surfaces grammatical metadata for every translated word; saved vocabulary feeds a SuperMemo-SM-2 flashcard scheduler; AI-generated CEFR-graded short stories are read aloud with synchronised highlighting via the Google Cloud Chirp 3 HD voices; and a basic community layer lets users share flashcard sets with a snapshot-fork model that preserves the original author's data.

Quantitatively, the system achieves corpus BLEU between 44.85 and 62.78 across its six translation directions on a Tatoeba-derived test set, with chrF++ between 62.53 and 72.92. The style classifier achieves an overall accuracy of 76.16 % and a macro-F1 of 0.77 across 432 evaluation sentences, with the most informative finding being the zero count for confusion between the formal and casual classes — the classifier knows where the poles are and only struggles to commit to non-neutral labels. The word-level analyser achieves 98 % POS accuracy and 88 % raw lemmatisation accuracy on a 100-entry Turkish test set, with the lemmatisation gap attributable mostly to a labelling-convention split that does not reflect genuine errors.

The limitations of this work are real and have been stated plainly: the translation evaluation does not directly test style targeting; the style evaluation uses LLM-generated test data; the word-analysis evaluation measures inter-model agreement rather than ground truth;

the system depends on a remote API for its register restyling; and the scope is limited to four languages and three register levels. None of these limitations is fatal, all of them are concrete enough to support future work, and several of them have been disclosed in this report at greater length than would be strictly required by the rubric — because a report that hides its weaknesses serves nobody, least of all its author.

Returning to where this report opened: I built Ivy because mainstream translators treat the choice of register as already made, and that is exactly the wrong default for someone who is still learning the language. A learner needs translation that is situated — that tells them not only what to say but how to say it appropriately to whom — and that comes embedded in an environment that helps them retain what they have learned. Ivy is one attempt to occupy that gap. It does not solve the problem; it shows that a meaningful piece of the problem can be addressed by a single student over a single semester, using freely available models, a thin layer of careful engineering, and an honest accounting of what was and was not measured. The harder work — replacing AI-on-AI evaluation with human gold standards, calibrating the classifier per language, building the parallel style-aware test set that would directly measure the pipeline's distinctive contribution — remains ahead, and is the work I would do next.

# ACKNOWLEDGEMENT

I would like to thank my supervisor, Asst. Prof. Ragıp Mustafa Başbuğ, for guidance and support throughout this capstone project, and the staff of the Department of Software Engineering at İstinye University for the academic environment that made this work possible. I also thank the maintainers of the open-source projects on which Ivy depends — Hugging Face Transformers, the Helsinki-NLP team, Flask, and many others — and the contributors to the Tatoeba and OPUS-100 corpora whose work made evaluation possible.

# APPENDIX A — PROMPT TEMPLATES AND KEY HYPERPARAMETERS

This appendix collects representative prompt templates and the most important hyperparameter settings used in the system, in the spirit of allowing a careful reader to reproduce the work.

## *A.1. Style-restyling prompt (Gemini, cross-lingual)*

```
Translate this {src_name} text into {tgt_name} with a {style} tone.  {rules}
CRITICAL RULES:   - Translate accurately — same meaning, nothing added, nothing
removed   - Do NOT add extra words, phrases, or politeness markers that are not in
the original   - Do NOT make the translation longer than necessary   - Only change
register: pronouns, verb conjugation, and word choice   - Output ONLY the
{tgt_name} translation — no quotes, no explanation  Text: {text}
```

Per-language register rules (the {rules} slot above) are inserted with explicit register markers; for Turkish these are the siz/sen distinction and the avoidance of formal copular suffixes in casual outputs.

## *A.2. Casual Turkish revise prompt (Gemini, single-language)*

```
Rewrite this Turkish sentence so it sounds like natural casual spoken Turkish.
Hard rules:   - Do NOT use the formal/written copula suffixes -dır, -dir, -dur, -
dür, -tır, -tir, -tur, or -tür.   - For nominal sentences like "X is Y", drop the
copula suffix or use a conversational phrasing.   - Keep the same meaning. Do not
add slang unless it is natural.   - Output ONLY the revised Turkish sentence.
Original source meaning: {source_text}  Turkish sentence to fix: {candidate}
```

## *A.3. Style-evaluation data generation prompt*

```
Generate {batch_size} distinct {lang_name} sentences for the following scenario:
Scenario: {scenario} Required register: {style} Register guidance: {hint}  Rules:
- Each sentence MUST be at most one sentence long (no compound multi-sentence
outputs).   - Each sentence MUST sound natural in {lang_name} — no awkward
translations from English.   - Sentences should vary in length and structure
within the batch.   - Output ONLY a JSON array of strings, like ["sentence one",
"sentence two", ...].   - Do not include explanations, numbering, or text outside
the JSON array.  Output:
```

Scenarios are sampled from a curated list per register (eight per register × three registers = 24 scenarios). The generation temperature is 0.85 (deliberately higher than the 0.2 used for

production restyling, to produce more lexical variation in the evaluation set), max output tokens 2048, thinking_budget = 0.

### *A.4. Key hyperparameters*

| **Component / Hyperparameter** | **Value** |
|---|---|
| Style classifier — base model | bert-base-multilingual-cased |
| Style classifier — epochs | 3 |
| Style classifier — optimiser | AdamW |
| Style classifier — learning rate | $2 \times 10^{-5}$ |
| Style classifier — batch size | 16 |
| Style classifier — weight decay | 0.01 |
| Style classifier — warmup | linear, first 10 % of steps |
| Style classifier — mixed precision | bfloat16 |
| Style classifier — training device | Google Colab A100 (40 GB) |
| Gemini — model | gemini-2.5-flash |
| Gemini — temperature (production restyling) | 0.2 |
| Gemini — temperature (eval data generation) | 0.85 |
| Gemini — max output tokens | 512 (restyling); 2048 (eval data) |
| Gemini — thinking_budget | 0 |
| Translation eval — sampling seed | 42 |
| Translation eval — sample size | 50 pairs per direction (300 total) |
| Translation eval — length filter | $5 \leq$ words $\leq 15$ |
| Style eval — sample size | 33 per (language, style) bucket + 6 seed entries (432 total) |
| Word analysis eval — sample size | 100 (Turkish, GPT-4-generated labels) |

# REFERENCES


[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)* (pp. 5998–6008).

[2] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015)*.

[3] Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems 27 (NeurIPS 2014)* (pp. 3104–3112).

[4] Sennrich, R., Haddow, B., & Birch, A. (2016). Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL 2016)* (Vol. 1, pp. 1715–1725). Berlin, Germany.

[5] Junczys-Dowmunt, M., Grundkiewicz, R., Dwojak, T., Hoang, H., Heafield, K., Neckermann, T., Seide, F., Germann, U., Aji, A. F., Bogoychev, N., Martins, A. F. T., & Birch, A. (2018). Marian: Fast neural machine translation in C++. In *Proceedings of ACL 2018, System Demonstrations* (pp. 116–121).

[6] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019* (Vol. 1, pp. 4171–4186).

[7] Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L., & Stoyanov, V. (2020). Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020)* (pp. 8440–8451).

[8] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research, 21*(140), 1–67.

[9] Xue, L., Constant, N., Roberts, A., Kale, M., Al-Rfou, R., Siddhant, A., Barua, A., & Raffel, C. (2021). mT5: A massively multilingual pre-trained text-to-text transformer. In *Proceedings of NAACL-HLT 2021* (pp. 483–498).

[10] Tiedemann, J. (2012). Parallel data, tools and interfaces in OPUS. In *Proceedings of the Eighth International Conference on Language Resources and Evaluation (LREC 2012)* (pp. 2214–2218).

[11] Tiedemann, J., & Thottingal, S. (2020). OPUS-MT — Building open translation services for the World. In *Proceedings of the 22nd Annual Conference of the European Association for Machine Translation (EAMT 2020)* (pp. 479–480). Lisbon, Portugal.

[12] Tiedemann, J., Aulamo, M., Bakshandaeva, D., Boggia, M., Grönroos, S.-A., Nieminen, T., Raganato, A., Scherrer, Y., Vazquez, R., & Virpioja, S. (2023). Democratizing neural machine translation with OPUS-MT. *Language Resources and Evaluation, 58*, 713–755.

[13] Shen, T., Lei, T., Barzilay, R., & Jaakkola, T. (2017). Style transfer from non-parallel text by cross-alignment. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)* (pp. 6830–6841).

[14] Hu, Z., Yang, Z., Liang, X., Salakhutdinov, R., & Xing, E. P. (2017). Toward controlled generation of text. In *Proceedings of the 34th International Conference on Machine Learning (ICML 2017)* (pp. 1587–1596).

[15] Li, J., Jia, R., He, H., & Liang, P. (2018). Delete, retrieve, generate: A simple approach to sentiment and style transfer. In *Proceedings of NAACL-HLT 2018* (Vol. 1, pp. 1865–1874).

[16] Rao, S., & Tetreault, J. (2018). Dear Sir or Madam, may I introduce the GYAFC dataset: Corpus, benchmarks and metrics for formality style transfer. In *Proceedings of NAACL-HLT 2018* (Vol. 1, pp. 129–140).

[17] Niu, X., Martindale, M., & Carpuat, M. (2017). A study of style in machine translation: Controlling the formality of machine translation output. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP 2017)* (pp. 2814–2819). Copenhagen, Denmark.

[18] Niu, X., Rao, S., & Carpuat, M. (2018). Multi-task neural models for translating between styles within and across languages. In *Proceedings of the 27th International Conference on Computational Linguistics (COLING 2018)* (pp. 1008–1021).

[19] Pavlick, E., & Tetreault, J. (2016). An empirical analysis of formality in online communication. *Transactions of the Association for Computational Linguistics, 4*, 61–74.

[20] Biber, D., & Conrad, S. (2019). *Register, genre, and style* (2nd ed.). Cambridge University Press.

[21] Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). BLEU: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL 2002)* (pp. 311–318).

[22] Popović, M. (2015). chrF: Character n-gram F-score for automatic MT evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation (WMT 2015)* (pp. 392–395).

[23] Sellam, T., Das, D., & Parikh, A. P. (2020). BLEURT: Learning robust metrics for text generation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020)* (pp. 7881–7892).

[24] Settles, B., & Meeder, B. (2016). A trainable spaced repetition model for language learning. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL 2016)* (Vol. 1, pp. 1848–1858).

[25] Woźniak, P. A., & Gorzelanczyk, E. J. (1994). Optimization of repetition spacing in the practice of learning. *Acta Neurobiologiae Experimentalis, 54*(1), 59–62.

[26] Eryiğit, G., Nivre, J., & Oflazer, K. (2008). Dependency parsing of Turkish. *Computational Linguistics, 34*(3), 357–389.

[27] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., … Amodei, D. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)* (pp. 1877–1901).

[28] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)* (pp. 24824–24837).